

# Layers and trust boundaries

One palace per process. Tenancy is vaults, not palaces. Nothing leaves the machine unless you configure it to.

## callers

agent over MCP studio  
33 tools · local process

app over HTTP /v1  
bearer token · REST

CLI  
remember · mine · search

admin console /ui  
served on every build

↓ bearer + optional per-vault assertion (HMAC, time-boxed)

## engine — one Rust process

/v1 surface + MCP handler  
drawers · search · refine · kg · verify  
rotate · export · import · taxonomy

store  
rank → gate → rerank → select  
verifies every row's HMAC before use

knowledge graph  
receipted facts, validity windows  
written only by refine or explicit kg calls

optional local LLM client  
Ollama or OpenAI-compatible, for refine  
unset MNEMOSYNE\_LLM\_URL ⇒ never called

## optional, separate binary

orchestrator  
tenant → vault routing, /t/\*  
own SQLite, sealed engine creds

remote vector backends  
untrusted accelerators — sealed  
payloads only, re-verified on read

↓ everything below this line is ciphertext

## at rest — per-vault key, HKDF-derived from the master key

sealed artifacts  
content · embeddings · ColBERT token  
matrices · PQ codes, codebook, pages  
each under a distinct AAD domain

integrity  
per-row HMAC tag, verified on every  
read; tamper-evident audit chain;  
manifest anchor as rollback detector

never present for sealed vaults  
no full-text index, no plaintext  
derived data in clear on disk.  
Search uses decrypt-once RAM caches.

Default posture: local-first and offline. No telemetry deps compiled in, no external API, deterministic embedder.

Each outward path is a deliberate opt-in: MNEMOSYNE\_LLM\_URL, MNEMOSYNE\_OTLP\_ENDPOINT, a remote index URL, MNEMOSYNE\_LLM\_KEY.